

Module 12 — Docker & Kubernetes

Interview-relevant containerization + orchestration for a Spring Boot service. Focus: efficient images, the core K8s objects, config/secrets, probes, and how a Spring Boot app is deployed and scaled.

12.1 Docker & Dockerfile

Core Concept

A **container** packages an app + its dependencies into an isolated, portable unit sharing the host kernel (namespaces + cgroups) — lighter than a VM (no guest OS). An **image** is the immutable, layered template; a **container** is a running instance.

Efficient Spring Boot Dockerfile (multi-stage)

```
# ---- build ----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -q -DskipTests package

# ---- run ----
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Best practices / layering

- **Multi-stage build** → small runtime image (JRE, not JDK).
- **Layer caching**: copy `pom.xml`/deps and resolve **before** copying source, so code changes don't re-download dependencies. Spring Boot **layered jars** / `bootBuildImage` (buildpacks) split deps vs app for better caching.
- Run as **non-root** user; pin base image; `.dockerignore`; small base (distroless/alpine caveats with glibc).
- Set memory-aware flags (`MaxRAMPercentage`) so the JVM respects cgroup limits.

ASCII — image layers

```
[ base JRE layer ] (rarely changes, cached)
[ dependencies layer ] (changes when pom changes)
[ application classes ] (changes every build)
-> only top layers rebuild/push
```

Interview Q / Follow-ups

- Container vs VM.
- Image vs container; what are layers; how does caching work.
- Why multi-stage builds; JDK vs JRE in the final image.
- How do you make the JVM respect container memory limits?

12.2 Docker Compose

Declaratively run multi-container local stacks (app + Postgres + Redis + Kafka) with one `docker compose up`. Defines services, networks, volumes, env, depends_on. Great for local dev/integration tests (Testcontainers uses similar ideas), **not** production orchestration.

Interview Q

When Compose vs Kubernetes; how services discover each other in Compose (service name DNS).

12.3 Kubernetes Core Objects

Why Interviewers Ask This

K8s is the de-facto production runtime. They want the core objects and how a Spring Boot app is deployed, configured, scaled, and exposed.

Objects

- **Pod** — smallest deployable unit; one or more containers sharing network/ storage; ephemeral (gets a new IP on reschedule).
- **ReplicaSet** — keeps N pod replicas running (usually managed by a Deployment).
- **Deployment** — declarative pods + rolling updates/rollbacks; the workload you usually create.
- **Service** — stable virtual IP + DNS load-balancing across pods. Types: **ClusterIP** (internal), **NodePort**, **LoadBalancer** (cloud LB).
- **Ingress** — HTTP(S) routing (host/path) + TLS into Services (needs an ingress controller, e.g. nginx).
- **ConfigMap** — non-secret config (env vars / mounted files).
- **Secret** — sensitive data (base64-encoded, not encrypted by default; enable encryption-at-rest / external secret managers).
- Also: **HPA** (Horizontal Pod Autoscaler by CPU/metrics), **StatefulSet** (stable identity/storage for stateful apps), **DaemonSet**, **Job/CronJob**, **Namespace**, **PV/PVC**.

ASCII — request path

```
Internet -> Ingress (host/path, TLS)
          -> Service (ClusterIP, DNS, LB across pods)
          -> Pod(s) [ Spring Boot container ]
ConfigMap/Secret -> env/volume -> app
HPA watches CPU/metrics -> scales Deployment replicas
```

Deployment lifecycle / probes

- **Rolling update**: new ReplicaSet scaled up while old scaled down (maxSurge/ maxUnavailable); readiness gates traffic; rollback with `kubectl rollout undo`.
- **Probes** (map to Actuator health groups):
- **liveness** → restart container if it hangs (`/actuator/health/liveness`).
- **readiness** → remove from Service until ready/deps ok (`/actuator/health/readiness`).
- **startup** → protect slow-starting JVMs from premature liveness kills.

Real Production Example

Spring Boot Deployment with 3 replicas, resource requests/limits, **ConfigMap** for `application.yml`, **Secret** for DB creds, readiness+liveness probes on Actuator, HPA scaling on CPU 70%, exposed via ClusterIP Service + Ingress with TLS.

Common Mistakes

- No resource requests/limits → noisy-neighbor / OOMKilled / scheduling issues.
- Liveness probe too aggressive → restart loops during slow startup (use startup probe).
- Storing secrets in ConfigMaps / images.
- Assuming pod IP/storage is stable (it isn't — use Services/PVCs).
- JVM not container-aware (old JDKs) → OOMKilled.

Interview Q / Follow-ups

- Pod vs Deployment vs Service vs Ingress.
- Service types (ClusterIP/NodePort/LoadBalancer).
- ConfigMap vs Secret; are Secrets encrypted?
- Liveness vs readiness vs startup probes.
- How does a rolling update / rollback work?
- How does HPA autoscale? How do pods find each other? (*Service DNS.*)

Hands-on Exercise

Write a Deployment + Service + Ingress + ConfigMap + Secret for a Spring Boot app; add liveness/readiness probes on Actuator; add an HPA; do a rolling update and a rollback.

Module 12 — One-Page Cheat Sheet

Object	Role
Pod	smallest unit; ephemeral; shares net/storage
ReplicaSet	maintains N replicas
Deployment	declarative pods + rolling update/rollback
Service	stable IP/DNS, LB across pods (ClusterIP/NodePort/LB)
Ingress	HTTP(S) routing + TLS
ConfigMap	non-secret config
Secret	sensitive data (base64; encrypt at rest)
HPA	autoscale replicas by metrics
Probes	liveness (restart), readiness (LB gate), startup (slow boot)

Docker: multi-stage, JRE runtime, layer caching, non-root, `MaxRAMPercentage`.

Module 12 — Top Interview Questions

1. Container vs VM; image vs container; layers & caching.
2. Why multi-stage builds; JDK vs JRE final image.
3. Pod vs Deployment vs ReplicaSet vs Service vs Ingress.
4. Service types and when to use each.
5. ConfigMap vs Secret; Secret security.
6. Liveness vs readiness vs startup probes (+ Actuator mapping).
7. Rolling update and rollback mechanics.
8. HPA autoscaling; requests vs limits.
9. Make the JVM respect container memory.
10. How do pods discover services?

Module 12 — Common Mistakes

- Shipping JDK / fat images; no `.dockerignore`.
- Missing resource requests/limits → OOMKilled/eviction.
- Aggressive liveness probe → restart loops.
- Secrets in images/ConfigMaps.
- Assuming stable pod IP/storage.

Module 12 — Mock Interview

1. “Your pod keeps restarting right after deploy.” → liveness probe firing during slow JVM startup; add a startup probe / raise initialDelay.
2. “Optimize a 700MB Spring image.” → multi-stage, JRE base, layered jar, `.dockerignore`, distroless.
3. “How does config differ per environment in K8s?” → ConfigMaps/Secrets per namespace + profiles; mount or env-inject.
4. “How does zero-downtime deploy work?” → rolling update gated by readiness probe; rollback via `rollout undo`.
5. “App gets OOMKilled though heap looks fine.” → total container memory (heap + metaspace + threads + native) exceeds the limit; set `MaxRAMPercentage` and right-size limits.

Next → Module 13: Production Scenarios (debugging).